Software Engineering I

Assignment 4 Analysis

Fabian Gonzalez

October 15, 2026

1. The two GenAI programs that I used for this assignment are ClaudeAI and Gemini.

2. I accessed both of them through the google search window. Both Gemini and Claude can be accessed by searching them up and locating them on the Google search engine.

3. Prompt used for both GenAI models:

(puzzle files are attached in the script)

Sudoku is a number puzzle in which a 9x9 grid is subdivided into 9 3x3 sub-grids, or blocks. Each of the cells in the puzzle must be filled in with a single digit 1-9, subject to the following constraints:
o Each digit from 1 to 9 must appear exactly once in each row.
o Each digit from 1 to 9 must appear exactly once in each column.
o Each digit from 1 to 9 must appear exactly once in each block.
You are given 5 puzzles files attached above. Digits already filled in have been placed, and (_) is used to indicate a blank square. Write a recursive object-oriented program in C++ using objects, not functions, to find the solution to the five puzzle files. Solve by depth-first search with backtracking. The output for each solution, should include:
o The puzzle file name (e.g., puzzle1.txt)
o The puzzle from the file printed out.
o The solution to the puzzle printed out or if no solution is found, “No solution found” printed out.
The solution may not be unique; if there is more than one solution to the puzzle, your program should print them all out.

4. Generated code from both GenAIs:

Claude:

```
// Sudoku solver: object-oriented, recursive depth-first search with backtracking.
// Prints every solution for each puzzle file given on the command line
// (defaults to puzzle1.txt ... puzzle5.txt).
#include <fstream>
#include <iostream>
#include <string>
#include <vector>

// A 9x9 grid of digits (0 = blank). Knows how to load, print and validate itself.

class Grid {

public:

    int cell[9][9];
```

```
Grid() { clear(); }

void clear() {
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++) cell[r][c] = 0;
}

bool load(const std::string& filename) {
    std::ifstream in(filename);
    if (!in) return false;
    clear();
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++) {
            std::string tok;
            if (!(in >> tok)) return false;
            cell[r][c] = (tok[0] >= '1' && tok[0] <= '9') ? tok[0] - '0' : 0;
        }
    return true;
}

void print(std::ostream& os = std::cout) const {
    for (int r = 0; r < 9; r++) {
        if (r % 3 == 0 && r > 0) os << "------+-------+------\n";
        for (int c = 0; c < 9; c++) {
            if (c % 3 == 0 && c > 0) os << "| ";
            if (cell[r][c]) os << cell[r][c] << ' ';
            else os << "_ ";
        }
```

```
        os << '\n';
    }
}

// Can digit d legally be placed at (r,c)?
bool canPlace(int r, int c, int d) const {
    for (int i = 0; i < 9; i++) {
        if (cell[r][i] == d) return false;
        if (cell[i][c] == d) return false;
    }
    int br = r / 3 * 3, bc = c / 3 * 3;
    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (cell[br + i][bc + j] == d) return false;
    return true;
}

// Are the given clues free of conflicts?
bool cluesConsistent() {
    for (int r = 0; r < 9; r++)
        for (int c = 0; c < 9; c++) {
            int d = cell[r][c];
            if (!d) continue;
            cell[r][c] = 0;
            bool ok = canPlace(r, c, d);
            cell[r][c] = d;
            if (!ok) return false;
        }
```

```
        return true;
    }
};

// Solver object: owns a working grid and collects every solution found by
// recursive DFS with backtracking.
class Solver {
public:
    explicit Solver(const Grid& start) : work(start) {}

    void run() {
        solutions.clear();
        if (work.cluesConsistent()) search(0);
    }

    const std::vector<Grid>& results() const { return solutions; }

private:
    Grid work;
    std::vector<Grid> solutions;

    // Recursive DFS over cells in row-major order; pos = 0..81.
    void search(int pos) {
        while (pos < 81 && work.cell[pos / 9][pos % 9] != 0) pos++;  // skip givens
        if (pos == 81) {                // every cell filled: record solution
            solutions.push_back(work);
            return;
        }
```

```
        int r = pos / 9, c = pos % 9;
        for (int d = 1; d <= 9; d++) {
            if (work.canPlace(r, c, d)) {
                work.cell[r][c] = d;    // choose
                search(pos + 1);        // explore
                work.cell[r][c] = 0;    // backtrack
            }
        }
    }
};

// Runs one puzzle file and reports on it.
class PuzzleReport {
public:
    explicit PuzzleReport(const std::string& f) : filename(f) {}

    void execute() {
        std::cout << "==================================================\n";
        std::cout << "Puzzle file: " << filename << "\n";
        Grid g;
        if (!g.load(filename)) {
            std::cout << "Could not read file.\n\n";
            return;
        }
        std::cout << "Puzzle:\n";
        g.print();

        Solver s(g);
```

```
        s.run();
        const std::vector<Grid>& sols = s.results();
        if (sols.empty()) {
            std::cout << "\nNo solution found\n\n";
            return;
        }
        std::cout << "\n" << sols.size() << " solution(s) found\n";
        for (size_t i = 0; i < sols.size(); i++) {
            std::cout << "\nSolution " << (i + 1) << ":\n";
            sols[i].print();
        }
        std::cout << "\n";
    }

private:
    std::string filename;
};

int main(int argc, char** argv) {
    std::vector<std::string> files;
    for (int i = 1; i < argc; i++) files.push_back(argv[i]);
    if (files.empty())
        for (int i = 1; i <= 5; i++) files.push_back("puzzle" + std::to_string(i) + ".txt");

    for (size_t i = 0; i < files.size(); i++) {
        PuzzleReport rep(files[i]);
        rep.execute();
    }
```

```
    return 0;
}
```

Gemini:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>

class SudokuBoard {
private:
    int grid[9][9];

public:
    SudokuBoard() {
        for (int i = 0; i < 9; ++i)
            for (int j = 0; j < 9; ++j)
                grid[i][j] = 0;
    }

    void loadFromFile(const std::string& filename) {
        std::ifstream file(filename);
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                char ch;
                if (file >> ch) {
                    if (ch == '_') {
                        grid[i][j] = 0;
```

```
                } else {
                    grid[i][j] = ch - '0';
                }
            }
        }
    }
}

void print() const {
    for (int i = 0; i < 9; ++i) {
        for (int j = 0; j < 9; ++j) {
            if (grid[i][j] == 0) std::cout << "_ ";
            else std::cout << grid[i][j] << " ";
        }
        std::cout << "\n";
    }
}

bool isValid(int row, int col, int num) const {
    // Check row
    for (int j = 0; j < 9; ++j) {
        if (grid[row][j] == num) return false;
    }
    // Check column
    for (int i = 0; i < 9; ++i) {
        if (grid[i][col] == num) return false;
    }
    // Check 3x3 block
```

```
        int startRow = row - row % 3;
        int startCol = col - col % 3;
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (grid[startRow + i][startCol + j] == num) return false;
            }
        }
        return true;
    }

    int getCell(int r, int c) const { return grid[r][c]; }
    void setCell(int r, int c, int val) { grid[r][c] = val; }
};

class SudokuSolver {
private:
    SudokuBoard board;
    std::string filename;
    int solutionCount;

    bool solve(int row, int col) {
        if (row == 9) {
            solutionCount++;
            std::cout << "Solution " << solutionCount << ":\n";
            board.print();
            std::cout << "\n";
            return false; // Continue searching for all possible solutions
        }
```

```
        int nextRow = (col == 8) ? row + 1 : row;
        int nextCol = (col == 8) ? 0 : col + 1;

        if (board.getCell(row, col) != 0) {
            return solve(nextRow, nextCol);
        }

        for (int num = 1; num <= 9; ++num) {
            if (board.isValid(row, col, num)) {
                board.setCell(row, col, num);
                solve(nextRow, nextCol);
                board.setCell(row, col, 0); // Backtrack
            }
        }
        return false;
    }

public:
    SudokuSolver(const std::string& fname) : filename(fname), solutionCount(0) {
        board.loadFromFile(filename);
    }

    void run() {
        std::cout << "Puzzle File Name: " << filename << "\n";
        std::cout << "Original Puzzle:\n";
        board.print();
        std::cout << "\n";
```

```
        solutionCount = 0;
        solve(0, 0);

        if (solutionCount == 0) {
            std::cout << "No solution found\n";
        }
        std::cout << "----------------------------------------\n\n";
    }
};

int main() {
    std::vector<std::string> files = {"puzzle1.txt", "puzzle2.txt", "puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};
    for (const auto& file : files) {
        SudokuSolver solver(file);
        solver.run();
    }
    return 0;
}
```

5. Comparing the two in terms of correctness, execution time, space complexity, and maintainability

Correctness:

Claude - Program correctly implements a depth-first search with backtracking. The program adheres to the rules of sudoku and ensures that there are no duplicates in each row or column of the 9x9 grid in each 3x3 block. However, one of its flaws is that the programming traverses in a fixed row-order skipping any givens. This can cause mistakes in the traversal order of the programming.

Gemini - Program correctly implements a depth-first search with backtracking. The program provides if each of the puzzles has a solution. Puzzles 1-3 have one solution each, Puzzle 4 has no solutions, and Puzzle 5 has two solutions. One of its flaws is that it cannot handle contradictory inputs like a smaller or bigger input file or invalidated characters.

Time Execution:

- Claude - Best-case scenario: O(1) . Worst-case scenario: O(9^E) or O(81)
- Gemini - Best-case scenario: O(1). Worst-case scenario: (27^9^k)

Claude and Gemini both have similar execution times for best-case scenarios. But in terms of worst-case scenarios, Gemini has the larger execution time based on if the sudoku board had very little to no digits on the board.

Space Complexity:

- Claude - Best-case scenario: O(1). Worst-case scenario: O(K)
- Gemini - Best-case scenario: O(1). Worst-case scenario: O(81)

Similar to the execution time, both Claude and Gemini have good best-case scenario times. Although it may look like they have different worst-case times as well, that does not seem to be the case. K in O(K) of Claude is the total number of solutions found. If K is a large number, then space complexity will increase linearly. In comparison to Gemini, space complexity is 81 because it goes through at most 82 frames (positions 0-81) before printing the solutions.

Maintainability:

Claude - Main weaknesses found in the program is that it lacks named constants which would give context to the number, and that the program contains tight coupling which would require rewriting to the overall program.

Gemini - Main weaknesses found in the program is that there is no error handling for contradictory information such as different sized grids, or unknown characters.

6. The program that I will use for the basis of my assignment will be the Gemini code. Despite its flaws, the program is short and more readable to the user in comparison to the Claude program. Additionally, the program already has good enough maintainability, aside from hardcoded files or numbers, and would likely need some light changing to make it more readable and overall manageable.

7. I improved the correctness of the code by adding in error handling into the program, which will now detect if the puzzle file has too many or too few rows or columns, and correctly passes the files that have the exact amount needed.
   I improved the execution time of the code by adding constraint checking in the program using bit-wise operations in order to make the program run at O(1) in both best and worst case scenarios.
   I improved the space complexity of the code by adding a limit parameter to determine how many solutions have been met and to prevent the program from running longer than necessary.
   I improved the maintainability of the code by better organizing the structure of the programming and returns any known errors for better user feedback.